In [5]:
```
# 1. Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import Flatten, Dense
```

In [6]:
```
# 2. Load both datasets
train_data = pd.read_csv("mnist_train.csv")
test_data = pd.read_csv("mnist_test.csv")

# 3. Separate images and labels
X_train = train_data.iloc[:, 1:].values
y_train = train_data.iloc[:, 0].values

X_test = test_data.iloc[:, 1:].values
y_test = test_data.iloc[:, 0].values

print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)
```

```
Training shape: (60000, 784)
Testing shape: (10000, 784)
```

In [7]:
```
# 4. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 5. Reshape images into 28x28 with 1 channel
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
```

In [9]:
```
# 6. Define CNN model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu',
           input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
```

```
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)	0
flatten_1 (Flatten)	(None, 5408)	0
dense_2 (Dense)	(None, 128)	692,352
dense_3 (Dense)	(None, 10)	1,290

Total params: 693,962 (2.65 MB)

Trainable params: 693,962 (2.65 MB)

Non-trainable params: 0 (0.00 B)

In [10]:
```
# 7. Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are ins
talled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

In [11]:
```
# 8. Train model
history = model.fit(
    X_train, y_train,
    epochs=5,
```

```
    batch_size=32,
    validation_split=0.1
)
```

```
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 32s 18ms/step - accuracy: 0.9527 - loss: 0.1592 - val_accuracy: 0.9820 - val_loss: 0.0659
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 58s 28ms/step - accuracy: 0.9832 - loss: 0.0547 - val_accuracy: 0.9858 - val_loss: 0.0500
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 71s 21ms/step - accuracy: 0.9886 - loss: 0.0356 - val_accuracy: 0.9875 - val_loss: 0.0501
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 39s 20ms/step - accuracy: 0.9928 - loss: 0.0227 - val_accuracy: 0.9860 - val_loss: 0.0583
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 38s 18ms/step - accuracy: 0.9952 - loss: 0.0156 - val_accuracy: 0.9875 - val_loss: 0.0553
```

In [12]:
```
# 9. Evaluate using the official test dataset
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")

# 10. Plot training and validation accuracy and loss
plt.figure(figsize=(12, 4))

# Accuracy graph
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

# Loss graph
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
```

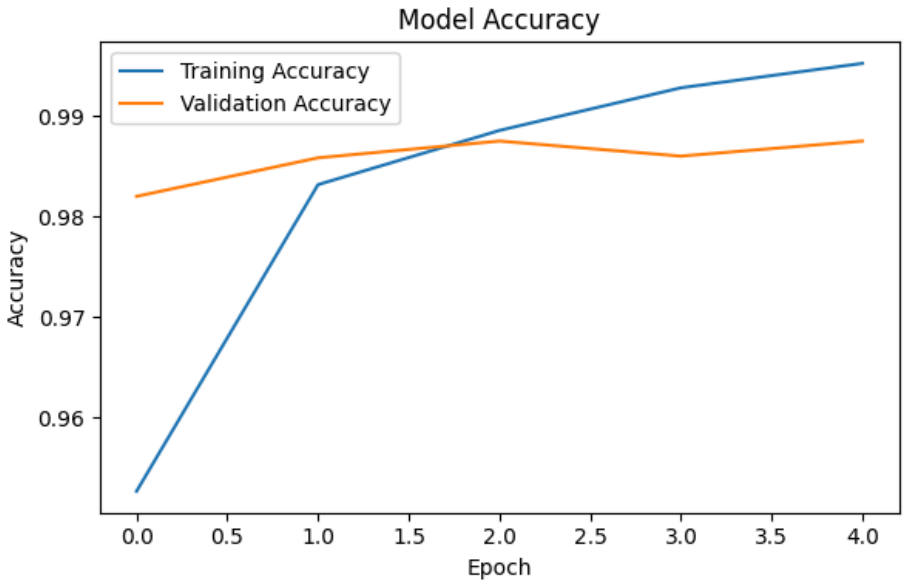
```
plt.tight_layout()
plt.show()

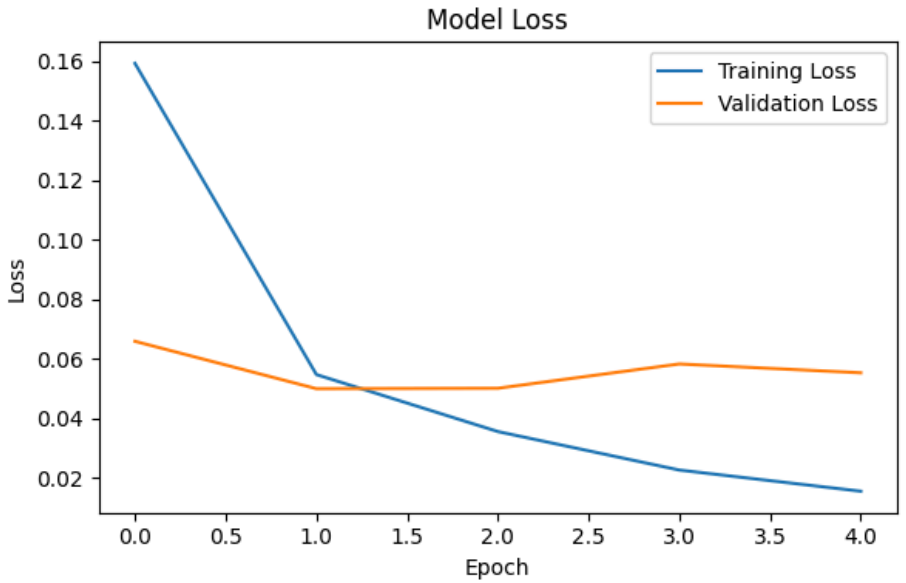
# 11. Predict a sample image
index = 0

prediction = model.predict(X_test[index:index+1])
predicted_digit = np.argmax(prediction)

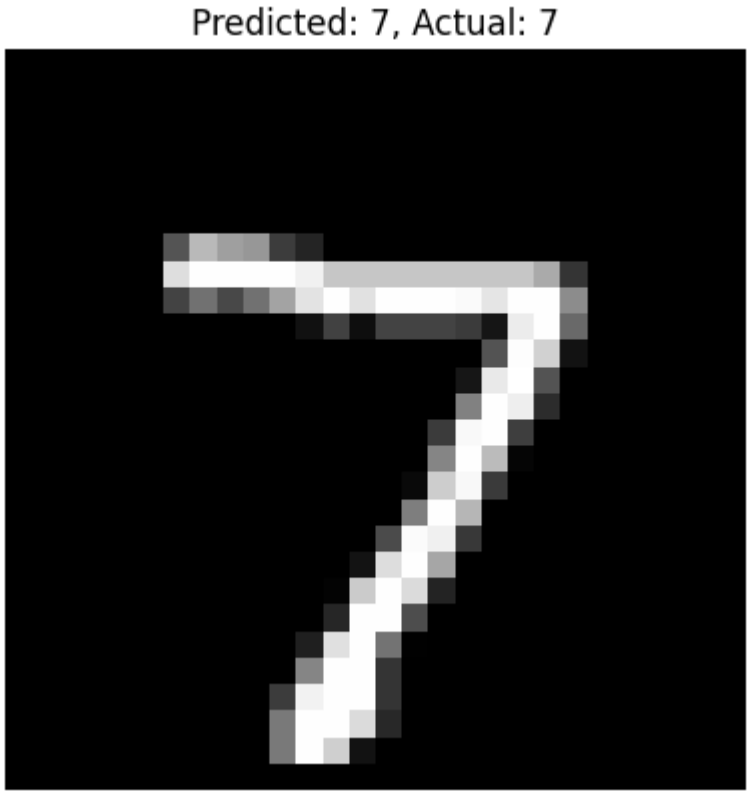
plt.imshow(X_test[index].reshape(28, 28), cmap="gray")
plt.title(f"Predicted: {predicted_digit}, Actual: {y_test[index]}")
plt.axis("off")
plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9874 - loss: 0.0426
Test Loss: 0.0426073856651783
Test Accuracy: 98.73999953269958 %
```






```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 168ms/step
```



In [ ]: